

End-to-End Automated UI Testing Workflow for Web Sites with Intensive User-System Interactions

Ramazan Faruk Oguz

*Computer Engineering Department,
Yildiz Technical University, Istanbul, Turkey
faruk.oguz@std.yildiz.edu.tr*

Izzettin Erdem

*Computer Engineering Department,
Yildiz Technical University, Istanbul, Turkey
izzettin.erdem@std.yildiz.edu.tr*

Erdi Olmezogullari

*Development Center,
Microsoft, Oslo, Norway
erdi.olmezogullari@ozu.edu.tr*

Mehmet S. Aktas

*Computer Engineering Department,
Yildiz Technical University, Istanbul, Turkey
aktas@yildiz.edu.tr*

Received (Day Month Year)

Revised (Day Month Year)

Accepted (Day Month Year)

The user interface (UI) is the software component end users interact with the most in the mobile and web world today. Therefore, when evaluating end-user satisfaction with a software, besides the functional features, the possibilities provided by the UIs are significant. End-user interfaces offer several capabilities today, necessitating a complex structure. Examples include interfaces developed in technologies such as Google Flutter and React. Therefore, we foresee a need for UI testing business workflows allowing comprehensive testing of UIs to improve customer satisfaction. Within the scope of this research, we propose an end-to-end approached automated UI testing business workflow for testing UI components on websites with intense user actions.

Keywords: automated UI testing; business workflow; end-user interface; comprehensive testing.

1. Introduction

Interest in sites with intensive user interaction, such as e-commerce sites, is increasing daily. However, the developments on these sites and the tests to be written

are gaining importance due to this interest. The comprehensiveness of such tests ensures the reliability of applications, thereby avoiding crashes. Therefore, the tests are gaining importance.

What do people look for when testing a site? Researchers and developers identify and test what business processes (activities) are to those who engage with sites the most. Moreover, with new demands from customers, regression tests are needed to analyze the requirements.

Currently, based on their background and experiences, test engineers manually create UI test scripts, using several testing tools (such as Selenium). In turn, this creates UI testing tasks that can provide a small coverage of UI functionalities and UI code. This is particularly the case with less-experienced test engineers, who tend to generate less comprehensive UI testing scripts. The focus of this study is creating a business workflow that can automate test script generation without relying on the test engineers' experiences or judgments. Hence, the aim is to generate UI test scripts that can provide high coverage testing.

The software development life cycle (SDLC) provides a perspective for the test phase. The tests that need to be written can be executed with a specific hierarchical sequence of steps: first, the analysis, then the design and coding, and finally, the test step. This study is concerned with ensuring the tests are done correctly and comprehensively. The SDLC involves the development of software in different categories, from architectural design to implementation, technical constraints to implementation, use cases to tests, and developer to end user. In this cycle, the importance of implementation and testing increases as the software matures to a certain extent. With the results obtained from the tests, the cycle continues, and improvements are made to the software.

Applications that are correctly tested and selected as broadly as possible are known to have fewer bugs, vulnerabilities, and issues. In addition, there are responsibilities that the software developer must undertake for the creation of these test flows. Although the size and complexity of the projects vary, tests are usually written right after the architectural design and the determination of the scenarios. Hence ensuring the acceleration and healthy progress of the software development process. Software, monolithic or not, starts to get complicated at a certain point in its life cycle. Testing at this point ensures that developers are least affected by complexity and improves the development process.

Today, several software projects appeal to the end-user, and with the most common example being e-commerce software. At this point, the front-end addressed the importance of the end-user gains. Nowadays, it is not enough for the front-end to look friendly to the user. However, developers are making an effort for a functionally effective front-end. At this point, the issue of applying software tests on the front-ends arises. TO ensure the continuous improvement of end users' experiences, eliminate security vulnerabilities, and ensure reliability, tests are run on front-ends, and the software is improved according to the test results.

Software projects that appeal to the end-user (such as e-commerce websites),

where the front-end gains importance, are constantly tested for the reasons mentioned above. The continuity of test processes and the growth of projects make it difficult to conduct front-end tests. These tests are usually performed by browsing the Uniform resource locators (URLs) of the products hosted by the websites. Different tools, such as Selenium and Cucumber, are currently used to automate test steps. URLs of web-based sites are required for the UI test scripts to be produced here. To generate these URLs, a particular workflow must be created.

The proposed methodology is designed to allow extensive testing of websites' t_0 immediately before end-users use them. Here, t_0 represents the website without any user interaction. Like the cold start problem in collaborative filtering for recommendation [1], it is important to thoroughly test websites first. There are different approaches to this problem in collaborative filtering [2, 3, 4], along with a literature review [5]. Regarding test script production, two different methods apply: user interaction for testing the website and testers implementing test scripts on the website according to their experiences. Our proposed end-to-end automated UI testing generation workflow offers a solution for this test script production situation.

The end-to-end automated UI testing generation workflow is designed to handle URL strings without any manual intervention. The proposed workflow uses the Generative Adversarial Network (GAN) approach, (a deep learning approach), as well as, it uses Hidden Markov Model (HMM) and Viterbi from statistical sciences. The proposed end-to-end automated UI testing generation workflow was successfully implemented and evaluated. The results show that URL sequences obtained can create test scripts to test a UI. The results of GAN and Viterbi, which are the sub-steps of the proposed workflow, also show that these methods can successfully produce URL sequences.

Motivation: Comprehensive creation of tests for a UI takes time for programmers. The URLs that makeup websites must be produced correctly in the test environment and run in the appropriate test steps. It is essential to reduce the effort here and automate the work. To meet this need, we have observed in the literature and to develop existing solutions, we have prepared a workflow for the automatic generation of test scripts. We propose a workflow starting from the website's home page, followed by automatically generating URL flows for test cases.

Research Problem: What should be a business process for UI testing that will allow the production of URL sequences that can be used for testing application UIs without the need for end-user interaction data? How should such a business process be developed and evaluated?

Within the scope of this research, we answer the following research questions regarding the research problem we have identified above.

- (1) Can UI tests are automated by creating an end-to-end workflow?
- (2) Can artificial URL sequences be created to test a newly created UI without user interaction data (click-stream)? (How can a site be tested at t_0 ?)
- (3) Can the successful creation of URL sequences designed for testing purposes be

measured?

- (4) Can the URL sequence generation approach be optimized when performing UI testing processes?

Contributions: Our study proposes an end-to-end automated UI testing generation workflow that can be automated for websites with heavy usage without prior user interaction. We claim that before a website meets its users, we can still create new test sequences for such an interface.

Organization: Section 2 presents the literature review on automated UI testing and deep learning approaches. Section 3 details the proposed methodology, while Section 4 describes the details of the proposed methodology's prototype and the evaluation of the proposed methodology. Lastly, Section 5 concludes the article and discusses future research directions.

2. Literature Review

An exploration of studies for automating the test steps in SDLC that there are few URL-based test generation flows and these studies do not meet an exact solutions for problems that we have introduced in the introduction [10, 16].

An examination of studies on mobile platforms for automated UI testing showed that user data is required and that user interactions are mocked and transferred to the test environment, again without being URL-based [9].

Some other studies have explored whether topic sequences can be tested by grouping URLs with regular expressions (regex), using the outputs here, and extracting the steps for the user to move forward [7]. A continuation of these studies presented methods that can be progressed without the need for user interaction [8].

In the literature, studies have attempted to identify user behaviors and produce specific patterns based on the click-stream navigations of users on websites [17, 18]. One of the common goals of these studies is to create clusters by using the patterns produced in different machine learning methods and to group similar users in these clusters. However, the focus of these studies has not been UI testing, and the outputs they produce are not available for UI testing. In our study, which is dissimilar to these studies, we propose an end-to-end automated UI testing generation workflow that can be used for UI testing and generates URL sequences that have not been encountered before, based on the click-stream navigation data made by users.

The HMM, which comprises patterns, observation states, latent states, and state distribution probabilities [19], is a statistical model primarily used in the fields of bioinformatics and speech recognition [20]. Different methods are used to calculate the possible situations in this model and to find the paths that can be taken. There are different areas where HMM is used. In their study, Imran et al. developed an HMM-based face recognition system consisting of four states [21]. Hajar et al. used an HMM to predict and detect False Data Injection attacks on Internet of Things systems [22]. Diksha et al. used an HMM for weather forecasting [23]. In their study, Ahmed et al. used an HMM to manage resource usage in cloud environments [24]. In

his study, Chao developed an HMM-based English speech recognition approach [25]. In the literature, there are different studies in which HMM is used, such as these studies. However, we did not come across any research suggested for the automatic generation of test scripts, as we have done in this study. Our aim is to address this deficiency in the literature.

Viterbi is an algorithm that finds the most probable path in the HMM based on a given observation sequence situation [26]. In our study, an HMM and Viterbi algorithms were used to generate the most likely URL sequences for the given topic sequences.

HMM can also be used to analyze user behavior. Kawazu et al. created an HMM using click-stream user navigation data and grouped user data [27], while Chuang et al. created an HMM-based model using user behavior data and calculated, using Viterbi, the probable user behaviors for a particular observation [28]. In contrast to these studies, we adopted the HMM to convert topic sequences to URL sequences within a testing business workflow.

There exist studies have analyzed large-scale user browsing data to extract useful knowledge [11, 12, 13]. There are also studies that focus on analyzing graph data to understand the data lineage [14, 15]. Contrastingly, in this study, we focus on analyzing graph data for test script generation. We also explored studies that have analyzed software quality by using static code analysis [30, 31]. In this study, we investigate the code quality by using UI testing techniques.

3. Proposed Methodology

We propose a business process that can automatically generate test scenarios that software developers can perform on a website without the need to use the relevant website and transfer them to Selenium. The flow of this business process is given in Figure 1.

3.1. *Crawling*

In this module, flows are extracted by navigating the website with random steps on the home page. The specific library is used inside the web crawler module to extract the streams, and its details are given in the prototype implementation step. At the end of this step, the structure of the website is created, the URL sequences that can be navigated are obtained, and data preprocessing was carried out by using this information in the next module.

3.2. *Regex Rule Assignment*

In this module, regex rules that will assign each URL to the appropriate topic are created by using the URL sequences taken from the crawling module and the structure of the website. In the next module, using these rules, the URLs found in the navigations of the users are mapped to the topics (Figure 2).

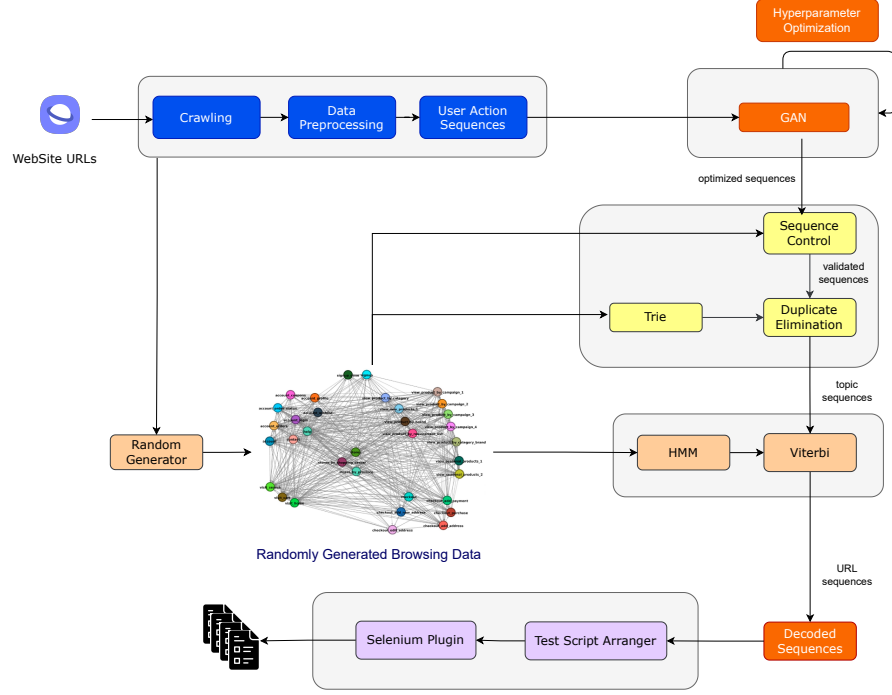


Fig. 1. Overview of the Proposed Workflow

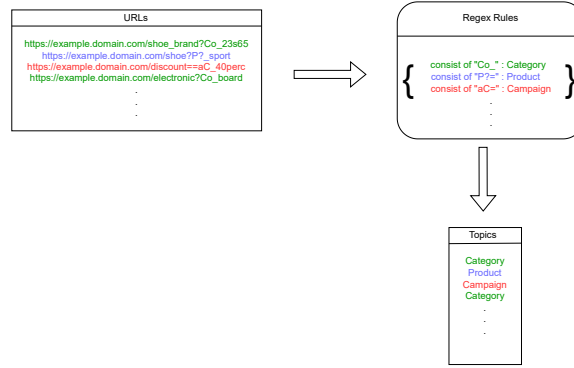


Fig. 2. Regex Rule Assignment Process

3.3. Data Preprocessing

This module uses user navigations of the users within the website as input. To obtain correct outputs from these sequences, each sequence is grouped according to userID, and the grouped sequences are ordered according to timestamp and filtered in such a way that only click data remains in the obtained dataset. Figure 3 shows

a brief illustration of the preprocessing steps.

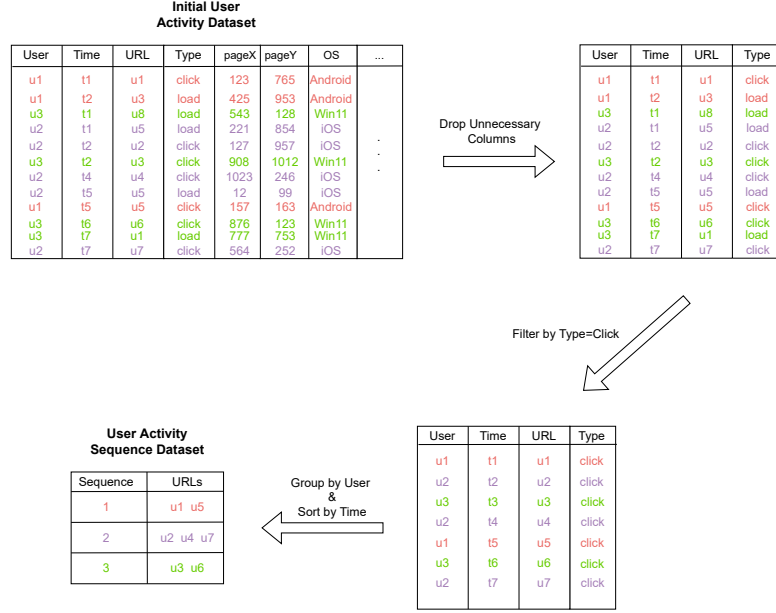


Fig. 3. Data Preprocessing

3.4. User Action Sequences

In this module, the regex rules created in the previous module are used on the dataset obtained in the previous module, and the URL sequences in the user navigations are mapped to the topics. At the end of the module, the topic sequences of the users' navigations are produced as output.

3.5. GAN

In this module, the GAN algorithm, a deep learning algorithm, is used to generate sequences. Since the GAN algorithm can produce output for a given input of a single type, the seqGAN algorithm, a type of GAN that can take the sequences in a dataset as input and produce sequences as output, is used in this module. In this algorithm, there are discriminator and generator classifiers, as in GAN [29].

For the seqGAN algorithm to work faster and more effectively, and to reduce complexity, each unique topic within the input topic sequences is mapped to numbers. The number of sequences produced as output are converted back to unique topic values after completion of the generation process.

The purpose of this module is to generate sequences that are similar to, but different from, the user sequences given as input. An illustration can be followed in Figure 4.

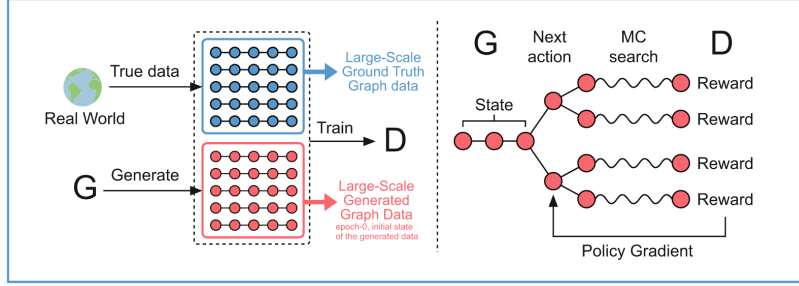


Fig. 4. Sequence Generative Adversarial Network

3.6. Hyperparameter Optimization

Hyperparameter optimization is required to improve the efficiency and evaluation of the produced topic sequences. Moreover, the parameters of deep learning algorithms must be optimized to be used with the best efficiency. Here, it is necessary to optimize three parameters for each iteration. The flow algorithm in the objective function can be followed from Algorithm 1.

3.7. Sequence Control

It is necessary to evaluate whether the optimized topic sequences are navigable or not, using the first dataset obtained by crawling beforehand. To do this, first, the sequences are produced and checked for a path between two points. An example of the steps described in the sequence control is depicted in Figure 5. Streams that are navigable are valid, and those that are not navigable are invalid sequences. Valid sequences are transferred to the next step.

3.8. Trie

A trie tree is created using the first dataset obtained with crawl so as to check and select new data that can be added to the tree (Figure 6). As can be followed in the figure, first, the topic sequence values are taken and then, they are put into the trie tree with the sequence mapper. The tree and sequence mapper data structures produced here will be used for the new topic sequences to be produced.

Algorithm 1 Objective Function**Require:** EMB_DIM, HID_DIM, GEN_SEQ_NUM**Ensure:** Perplexity

```

1: generated_sequences  $\leftarrow$  seqGAN(emb_dim, hid_dim, gen_seq_num)
2: training_sequences  $\leftarrow$  fileRead(train_seq)
3: encoded_training_sequences  $\leftarrow$  []
4: encoded_gnrtd_sequences  $\leftarrow$  []
5: for train_sequence in training_sequences do
6:   encoded_training_sequences.append(encode(train_sequence))
7: end for
8: for gen_sequence in training_sequences do
9:   encoded_gnrtd_sequences.append(encode(gen_sequence))
10: end for
11: trie  $\leftarrow$  Trie(encoded_training_sequences)
12: trie.fit()
13: encoded_unq_seqs, encode_dupl_seqs  $\leftarrow$  trie.check(encoded_gnrtd_sequences)
14: sequenceMap  $\leftarrow$  SequenceMap(encoded_training_sequences)
15: sequenceMap.fit()
16: enc_valid_seqs, enc_invalid_seqs  $\leftarrow$  sequenceMap.check(encoded_unq_seqs)
17: perplexity  $\leftarrow$  calcPerplexity(encoded_training_sequences, enc_valid_seqs)

```

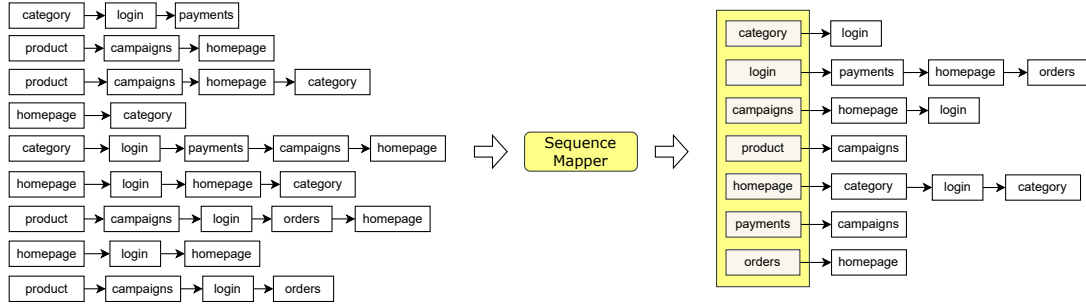


Fig. 5. Sequence Map

3.9. Duplicate Elimination

The validated sequences from the sequence control step are given to the tree created with the trie module, and the duplicate topic sequences are removed from the section where they were generated. This module produces navigable and singular streams that have not been seen before.

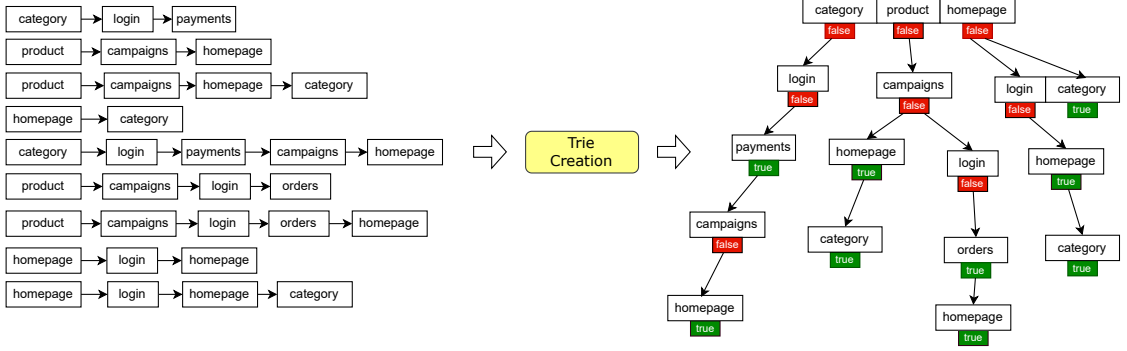


Fig. 6. Trie Tree

3.10. *HMM*

The transition and emission matrices required for the HMM are calculated in this module. First, the user URL navigation sequences obtained in the data preprocessing module and the user topic sequences obtained in the user action sequences module are used. Second, a dictionary is created from the unique data in the URL sequences, a number is assigned to each URL, and the same process is applied to the topic sequences. The transition and emission matrices are initialized using these dictionaries. If N is the total count of unique URLs and M is the total count of unique topics, the size of the transition matrix to be created will be $N \times N$, and the size of the emission matrix will be $M \times N$.

Here, URLs represent hidden states, and topics represent observation states. URL transitions for each user's navigation are examined, the number equivalents of these URLs in the dictionary are determined, and the value of the cell corresponding to these numbers in the transition matrix is increased by 1. Afterward, the transition matrix is created by calculating the probabilities of the values in the matrix.

Similarly, the URL navigations of the users are examined, and the related topics are found. The number values corresponding to these URL-topic binaries are found in the dictionaries, and the cell corresponding to these numbers is increased by 1 in the emission matrix. Then the emission matrix is created by calculating the probabilities of the values in the matrix. Finally, the stationary distribution is calculated using the transition matrix. This module takes URL and topic navigation sequences made by real users as input and produces an HMM module as output. The HMM creation steps can be followed in Figure 7.

3.11. *Viterbi*

The Viterbi algorithm finds the most probable hidden sequence for the given observation sequence by maximizing the finite probability, and it was used to calculate the path that can be reached from these inputs with the highest probability. This

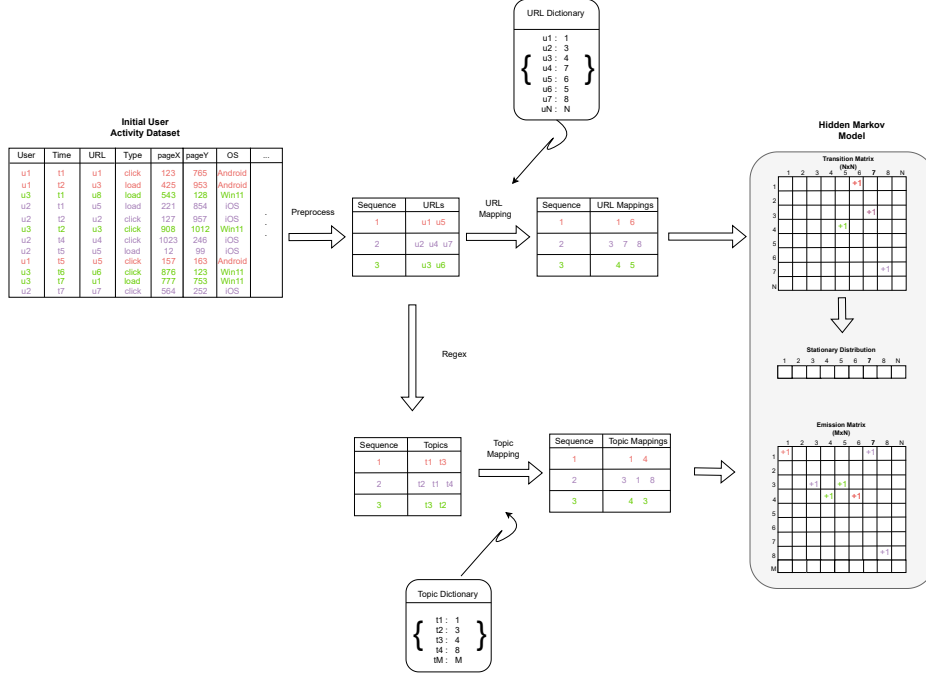


Fig. 7. Overview of the Hidden Markov Model

module generates URL sequences as output for the given topic sequences by using the probabilities in the HMM model and Viterbi algorithm.

HMM and Viterbi's primary purpose is not to create new artificial URL sequences but to find the most suitable URL sequences for newly released sequences.

In our study, the observation sequences were the topic sequences we produced with GAN; artificial sequences were URL sequences that are decoded with Viterbi and produced as output. Figure 8 shows a brief explanation of generated topic sequences' decoding process by using the Viterbi algorithm.

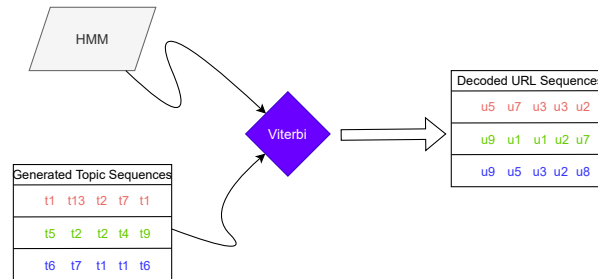


Fig. 8. Decoding URL Sequences by Using Viterbi Algorithm

3.12. *Test Script Arranger*

This module makes the URL sequences taken from the Viterbi module understandable in the Selenium plugin and gives it to the next step. The purpose of this module is to format the decoded sequences.

3.13. *Selenium Plugin*

An end-to-end automated UI test generation is completed by running artificial user URL navigations in this module with the formatted data taken from the previous module. Here, the flow is completed with the data coming directly from the test script arranger without using user data.

4. Prototype Implementation, Experiments and Evaluations

4.1. *Prototype Implementation*

In this study, a prototype was developed by using the Python programming language to implement the proposed business workflow. In the implementation, diverse open-source Python libraries were used. Scrapy (version 2.5.1) was used in the crawling module; regex (version 2021.4.4) in the regex rule assignment module; numpy (version 1.21.2) and pandas (version 1.2.4) for calculations in the GAN, HMM, Viterbi and test script arranger Modules; tensorflow (version 2) and nltk (version 3.5) in data preprocessing and GAN modules; and pickle for object serialization. There are many different algorithms for hyperparameter optimization [32, 33, 34]. In this study, tree of parzen estimator is used for algorithm and the `uniform_int_distribution` function for choosing the values of variables.

4.2. *Datasets*

In this study, we used data from three different websites, and we show the sample working flow of our system using these data. The first of these datasets is an e-commerce site selling mother and baby products. The other two datasets are Loadium and Testinium datasets, which also belong to Testinium company.

4.3. *Test Design*

To implement the proposed workflow, it is necessary to obtain the structure of the website of interest. A crawler, enables both obtaining the website structure and extracting the user flows that can be an example. In this way, the process is also done to generate the flows that users can go to. In the crawler implementation, scrapy was used, and the structure of the website and the URL sequences were found.

After the URL streams were found, the obtained data were preprocessed and used in different steps for easier understanding. These preprocesses and steps are outlined below:

- Determine the parts of the data that are advanced by click,
- Determine regexes,
- Apply the regex rules for the user action sequence,
- Generate new topic sequences by giving the obtained topic sequences to the GAN algorithm,
 - Calculate the transition, emission matrices, and stationary distribution required for the HMM using user action sequences,
 - Reproduce the top 500 URL sequences in user action sequences using HMM and Viterbi algorithms and measure the success of the Viterbi algorithm,
 - Calculate the similarities of topic sequences produced with GAN and topic sequences in user action sequences,
 - Generate URL sequences using Viterbi and the HMM from topic sequences produced with GAN,
 - Prepare test scripts that can be used in tools such as Selenium and Cucumber as it progresses.

Deep learning was run on topic sequences taken as input with seqGAN, which is a GAN type. As it is known, GAN flow contains two different networks in itself. These are the discriminator and generator. It is used to create long short-term memory (LSTM) and convolutional neural network (CNN) discriminator and generator networks in seqGAN, respectively. Of course, due to the nature of the GAN, the generator produces streams that will be very close to reality while the discriminator tries to classify these generated streams.

We tried to optimize three of the parameters used in the discriminator and generator, which we used here in our study. These are briefly the number of sequences to be produced, the embedding dimension used in LSTM and CNN, and the hidden dimension parameters. In hyperparameter optimization, 100 trials were made for each dataset, and the EMB_DIM and HIDDEN_DIM dimension parameters used in the generator and the number of sequences to be produced were tried to be optimized. Although the trial spaces are the same for each site, the resulting values differ in the sites.

The perplexity score is selected in the objective function to be used for optimization. Streams that are optimized and give the lowest perplexity score pass to the control section as optimized sequences. After running Trie and SequenceMap, both perplexity and bleu score values are calculated as 4-grams.

Transition and emission matrices were created using user-action sequences to calculate the probability matrices and distributions required for the HMM. The stationary distribution was calculated over the created emission matrix. Before proceeding to the URL sequence generation step, the topic sequences of the 500 most frequently mentioned URL sequences among the user-action sequences were found, and new URL sequences were obtained from these topic sequences by using the HMM and Viterbi algorithm. A one-to-one comparison was made between these obtained URL sequences and the 500 most frequent URL sequences, and the different ones were marked.

Since the Viterbi algorithm guarantees us to find the most probable path, the differences between URL sequences mean that there are more possible URL sequences that represent the topic sequences of the URL navigations users have taken so far. Therefore, future users are more likely to proceed through these URL sequences. The importance of this inference for us is that the test scripts can be produced using the Viterbi algorithm and HMM to represent future user behaviors so that the generated test scripts can enable us to anticipate the problems we may encounter in the future. Our results show us the success of HMM and Viterbi algorithm in a more concrete way. Figure 9 briefly shows the testing operation.

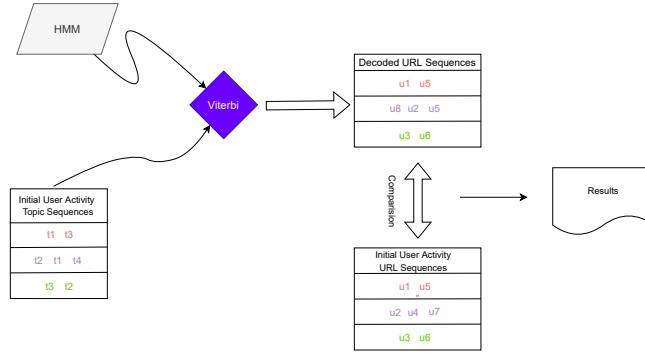


Fig. 9. Testing Process

After this calculation, a similarity calculation was made using edit distance between the unprecedented topic sequences produced using GAN in the previous steps and the topic sequences belonging to real users. We do this to answer the question of how many changes we can make to the topic sequences produced, and we can obtain the topic sequences visited by real users.

After these processes, the topic sequences we produced with GAN and the HMM were given input to the Viterbi algorithm, and URL sequences were produced. These URL sequences produced in the last step were converted into test scripts and used on test tools such as Selenium or Cucumber.

4.4. Experiments and Evaluation

After the proposed system was realized as a prototype, trials were carried out. Here, EMB_DIM and HIDDEN_DIM parameters used in the datasets are between 2^3 and 2^{15} , and the number of sequences to be produced was between 10000 and 1 million.

The first selected dataset is from an e-commerce site that generally markets mother and baby related products. Table 1 presents the perplexity and bleu score values measured in 4-grams, while Table 2 shows the best parameters produced for the dataset. In the tables, n-grams are adjoining sequences of n items from a given test sequence.

Table 1. Best Scores for Mother&Baby Products Site

	1-gram	2-gram	3-gram	4-gram
Bleu Score	0.966	0.857	0.776	0.711
Perplexity	6.772	7.629	5.994	4.790

Table 2. Best Parameters for Mother&Baby Products Site

	EMB_DIM	HIDDEN_DIM	Generated Sequence Num
Optimized Parameter	64	4096	165616

The second data used is the dataset of the Loadium site. Loadium is actually a performance testing and load testing tool and its main focuses are performance, load, stress and spike tests. Here, the operations applied in the first dataset were applied on the dataset, and then the results were run. The point that draws attention here is that the results in Table 3 are not similar to the results for the mother and baby products site. This is because the training dataset used was diverse and did not contain much data. The best parameters of the relevant site can be followed in Table 4.

Table 3. Best Scores for Loadium Site

	1-gram	2-gram	3-gram	4-gram
Bleu Score	0.882	0.728	0.59	0.452
Perplexity	8.437	8.672	6.493	5.042

Table 4. Best Parameters for Loadium Site

	EMB_DIM	HIDDEN_DIM	Generated Sequence Num
Optimized Parameter	2048	64	132075

The last dataset used in our research is the Testinium dataset which contained the navigations of the tests carried out by the company and the test categories applied. For this dataset, we obtained results similar to those of the first dataset, as there are also extensive and detailed user action sequences in the third dataset. The reason for the results obtained in Tables 3 and 4 related to Loadium can also be associated with this situation. We have verified and confirmed that data must contain several and multiple streams to achieve these results. The best parameters and evaluation results of the relevant site can be followed in Tables 5 and 6.

Different from the above, we made an edit distance calculation for each dataset between the topic sequences we produced, and the topic sequences of the tours made

Table 5. Best Scores for Testinium Site

	1-gram	2-gram	3-gram	4-gram
Bleu Score	0.979	0.859	0.777	0.701
Perplexity	6.256	7.332	5.988	4.892

Table 6. Best Parameters for Testinium Site

	EMB_DIM	HIDDEN_DIM	Generated Sequence Num
Optimized Parameter	32768	512	304761.0

by real users. According to the results we obtained, we can obtain topic sequences made by real users with an average of three changes on most of the topic sequences we produced for the first dataset, an average of three changes for the Loadium dataset, and an average of three changes for the Testinium dataset. This means that although the topic sequences we produce are different from the actual topic sequences, they are similar enough.

In Figure 10, we have three charts. In the first chart, it is seen that 18% of the produced topic sequences can be converted with a single change, on 55% with two changes, on 25% with three changes, on 2% with the four changes, and so on to their original form which are topic sequences navigated by real users.

In the second chart, it is seen that approximately 4% of the produced topic sequences can be converted with a single change, on 25% with two changes, on 48% with three changes, on 18% with the four changes and so on to their original form which are topic sequences navigated by real users.

In the third chart, it is seen that approximately 16% of the produced topic sequences can be converted with a single change, on 41% with two changes, on 34% with three changes, on 8% with the four changes and so on to their original form which are topic sequences navigated by real users.

These results show that the newly generated URL sequences are in fact similar (distribution probability-wise) to URL sequences that are used as input by the proposed workflow. The results indicate that by using the proposed workflow, we can create new URL sequences can be used in UI test scripts. Detailed information is shown in Figure 10. These charts show the edit distance between the topic sequences we produced, and the topic sequences navigated by real users. The first chart shows the Mother & Baby products dataset, the second chart shows the Loadium dataset, and the third chart shows the Testinium dataset.

In the last part of our experimental study, we produced new URL sequences by giving the topic sequences, and HMM we produced to the Viterbi algorithm, and we produced test scripts that can be given as input to tools such as Selenium and Cucumber. Since the success of the Viterbi algorithm could not able to measure with the accuracy metric, which is the number of correct predictions in the total number

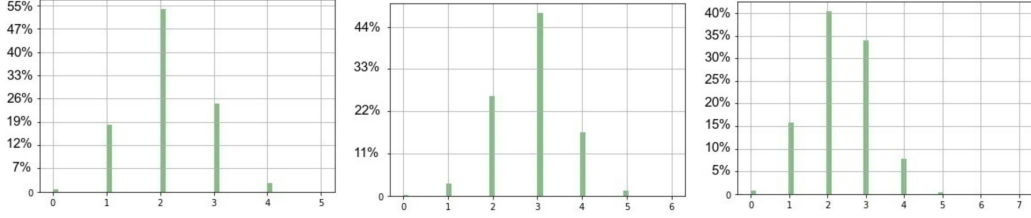


Fig. 10. Edit Distances. In each chart, x-axis means shows the percentage in the interested dataset, y-axis means specifies the edit distance value.

of predictions, the 500 topic sequences that are repeated the most by the users in each dataset were given to the Viterbi algorithm as the observation sequence, and a comparison with the real URL sequences was made by generating new URL sequences.

The results show that for each dataset we use, there are different URL sequences that better represent the 500 topic sequences we have. The difference between these different URL sequences produced by the Viterbi algorithm to the actual URL sequences is 0% on average. That is, for three out of every five trips a user has taken, different URL sequences are more likely to go. The results are detailed in Table 7.

Table 7. Differences between actual and generated URL Sequences

	Mother & Baby Products	Loadium	Testinium
Same	95	275	213
Different	405	225	287
Differences (%)	81	45	57

Discussion: Our previous work proposed a method that generates unprecedented URL sequences from user click-stream navigation data using HMM and Viterbi algorithms. In the calculations we made to measure the success of the HMM and Viterbi algorithms in that study, we calculated the similarities between the topic sequences produced using GAN and given as input and the URL sequences produced by the topic sequences [6]. We have seen that this calculation is correct but incomplete. In the current study, we changed the success criteria. We produced new URL sequences by giving the topic sequences of click-stream navigations made by real users to the HMM model we produced and the Viterbi algorithm. We compared these URL sequences with the URL sequences of the topic sequences we gave as input. Since the Viterbi algorithm produced the most probable path, we interpreted the difference between our output and the actual data as follows: The most likely URL sequences better represent the topic sequences of the users' trips so far and have not been visited before. So, using the Viterbi algorithm, we can find the

most probable user navigation sequences. In our current study, our success criteria was directly proportional to the difference between the sequences produced by the Viterbi algorithm and the sequences of real users.

The main purpose of this study was to propose a workflow that can generate test scripts in an automated way and, our aim in the evaluation of this study was how well the proposed business workflow generates new test scripts. To do this, we compared and contrasted the output URL sequences (used in new test scripts) and randomly generated URL sequences (used within the proposed workflow). We investigate the similarity between the two URL sequences. We argue that the high similarity between the distributions of two sequences indicate the success of the proposed workflow in generating the new URL sequences used in new test scripts. In this section, we discuss our answers to the research questions we posed in the Introduction section.

Answer to Research Question 1: In Figure 1, we illustrate the automated process that performs end-to-end UI testing.

Answer to Research Question 2: As we mentioned in the crawling module, after the basic structure of the website is obtained, user flows can be created on this structure and the workflow can be continued with the remaining steps through the flow that we proposed.

Answer to Research Question 3: To measure the end-to-end automated UI testing generation workflow, perplexity and blue score values were used, and the difference between topic sequences was also examined using edit distance.

Answer to Research Question 4: The workflow we propose has a structure that can learn within itself and convert what it has learned into output as an artificial URL sequence. This structure uses seqGAN, one of the deep learning algorithms in itself. Deep learning algorithms, of course, contain many parameters. In this study, we tried to optimize the three parameters of the deep learning algorithm we used in the hyperparameter optimization module.

Threats to Validity of Results: In this study, we want to show that we can calculate the most probable URL navigations of future users from the URL sequences we have created based on the navigations of real users on a site, by using the HMM and Viterbi algorithms, and at the same time, we want to expand the scope of the test scripts we will create using these calculations.

The results we obtained in this study depend on the website URL structure of the websites we have carried out. However, the proposed end-to-end automated UI testing business workflow that we have used constitutes a step-by-step mechanism independent of data. Moreover, we have conducted our experiments on three different websites to make sure that our introduced workflow is accurate. The proposed workflow is also detailed and described above and can be applied to different datasets.

5. Conclusion

In the study, we proposed an end-to-end automated UI testing generation workflow that can generate automatic UI test scripts on websites that have not been used before or are less used. We tested this recommended business workflow on three different sites. We believe that the results will shed light on future studies.

The designed business workflow creates unprecedented artificial URL sequences by using seqGAN from GAN variations of deep learning approaches, the tree of parzen estimator algorithm for hyperparameter optimization, and HMM and Viterbi from statistical methods.

In the seqGAN step, we used the core idea of GAN to produce new artificial sequences. To summarize, our primary throughput is new topic sequences based on indirect training through the discriminator and using another neural network: the generator.

In the HMM and Viterbi steps, our main motivation is to create sequences by mapping URLs visited by real users to topics and to produce URL sequences with the highest probability by calculating the transitions between these sequences. Thus, we are expanding the scope of the test scripts we will create to include the most likely navigations that future users can make.

We note that word embedding approaches should be used for topic sequences used in the later stages of our study. By optimizing more parameters in hyperparameter optimization, the study can yield better results.

As observed in the tests, we conducted using the end-to-end automated UI testing generation workflow, we recommend the use of HMM and Viterbi in test script generation, as they produce successful results. In our previous work, we stated that we wanted to use different datasets in a future study [6]. In this study, we achieved this goal by testing HMM and Viterbi on three different datasets in test script generation. In our future works, we want to focus on test script generation by using different methods besides HMM and Viterbi.

Acknowledgments

We are grateful to TUBITAK for supporting this research (under project id: 3191534) and Saha Bilgi Teknolojileri Egitim Danismanlık San. Tic. A.S. R&D Center for providing the computational materials that made this study achievable. Erdi Olmezogullari conducted this work while he was working in R&D Center at Testinium, Istanbul.

References

- [1] Lika B, Kolomvatsos K, Hadjiefthymiades S. Facing the cold start problem in recommender systems. *Expert systems with applications*. 2014 Mar 1;41(4):2065-73.
- [2] Zarei MR, Moosavi MR, Elahi M. Adaptive trust-aware collaborative filtering for cold start recommendation. *Behaviormetrika*. 2022 Mar 14:1-22.
- [3] Wei J, He J, Chen K, Zhou Y, Tang Z. Collaborative filtering and deep learning

- based hybrid recommendation for cold start problem. In 2016 IEEE 14th Intl Conf on Dependable, Autonomic and Secure Computing, 14th Intl Conf on Pervasive Intelligence and Computing, 2nd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech) 2016 Aug 8 (pp. 874-877). IEEE.
- [4] Wei T, Wu Z, Li R, Hu Z, Feng F, He X, Sun Y, Wang W. Fast adaptation for cold-start collaborative filtering with meta-learning. In 2020 IEEE International Conference on Data Mining (ICDM) 2020 Nov 17 (pp. 661-670). IEEE.
 - [5] Panda DK, Ray S. Approaches and algorithms to mitigate cold start problems in recommender systems: a systematic literature review. *Journal of Intelligent Information Systems*. 2022 Apr 23:1-26.
 - [6] Erdem, I., Oguz, R.F., Olmezogullari, E. and Aktas, M.S., 2021, December. Test Script Generation Based on Hidden Markov Models Learning From User Browsing Behaviors. In 2021 IEEE International Conference on Big Data (Big Data) (pp. 2998-3005). IEEE.
 - [7] Oz, M., Kaya, C., Olmezogullari, E. and Aktas, M.S., 2021. On the use of generative deep learning approaches for generating hidden test scripts. *International Journal of Software Engineering and Knowledge Engineering*, 31(10), pp.1447-1468.
 - [8] Oguz, R. F., Oz, M., Olmezogullari, E., & Aktas, M. S. (2022). Extracting information from large scale graph data: Case study on automated ui testing. In *European Conference on Parallel Processing* (pp. 364-375). Springer, Cham.
 - [9] Kaasila, J., Ferreira, D., Kostakos, V. and Ojala, T., 2012, December. Testdroid: automated remote UI testing on Android. In *Proceedings of the 11th International Conference on Mobile and Ubiquitous Multimedia* (pp. 1-4).
 - [10] Ki, T., Simeonov, A., Park, C.M., Dantu, K., Ko, S.Y. and Ziarek, L., 2017, June. Fully automated ui testing system for large-scale android apps using multiple devices. In *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services* (pp. 185-185).
 - [11] Olmezogullari, E., & Aktas, M. S. (2022). Pattern2Vec: Representation of clickstream data sequences for learning user navigational behavior. *Concurrency and Computation: Practice and Experience*, 34(9), pp. e6546.
 - [12] Olmezogullari, E. and Aktas, M.S., Representation of click-stream data sequences for learning user navigational behavior by using embeddings, 2020 IEEE International Conference on Big Data (Big Data), pages:3173-3179, 2020.
 - [13] Uygun, Y., Oguz, R.F., Olmezogullari, E. and Aktas, M.S., On the large-scale graph data processing for user interface testing in big data science projects, 2020 IEEE International Conference on Big Data (Big Data), pages: 2049-2056, 2020.
 - [14] Tufek, A., Gurbuz, A., Ekuklu, O. F., & Aktas, M. S. (2018, September). Provenance collection platform for the weather research and forecasting model. In 2018 14th International Conference on Semantics, Knowledge and Grids (SKG) (pp. 17-24). IEEE.
 - [15] Baeth, M. J., & Aktas, M. S. (2019). Detecting misinformation in social networks using provenance data. *Concurrency and Computation: Practice and Experience*, 31(3), pp. e4793.
 - [16] Arnatovich, Y.L., Ngo, M.N., Kuan, T.H.B. and Soh, C., 2016, December. Achieving high code coverage in android ui testing via automated widget exercising. In 2016 23rd Asia-Pacific Software Engineering Conference (APSEC) (pp. 193-200). IEEE.
 - [17] Su, Q. and Chen, L., 2015. A method for discovering clusters of e-commerce interest patterns using click-stream data. *electronic commerce research and applications*, 14(1), pp.1-13.

- [18] Ting, I.H., Kimble, C. and Kudenko, D., 2005, September. UBB mining: finding unexpected browsing behaviour in clickstream data to improve a Web site's design. In *The 2005 IEEE/WIC/ACM International Conference on Web Intelligence (WI'05)* (pp. 179-185). IEEE.
- [19] Yusoff, M.I.M., Mohamed, I. and Bakar, M.R.A., 2014, November. Hidden markov models: An insight. In *Proceedings of the 6th International Conference on Information Technology and Multimedia* (pp. 259-264). IEEE.
- [20] Deng, X. and Ali, H., 2004, August. A hidden markov model for gene function prediction from sequential expression data. In *Proceedings. 2004 IEEE Computational Systems Bioinformatics Conference, 2004. CSB 2004.* (pp. 670-671). IEEE.
- [21] Ali, D., Touqir, I., Siddiqui, A. M., Malik, J., & Imran, M. (2022). Face Recognition System Based on Four State Hidden Markov Model. *IEEE Access*, 10, pp. 74436-74448.
- [22] H. Moudoud, Z. Mlika, L. Khoukhi and S. Cherkaoui, "Detection and Prediction of FDI Attacks in IoT Systems via Hidden Markov Model," in *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 5, pp. 2978-2990, 1 Sept.-Oct. 2022.
- [23] D. Khiatani and U. Ghose, "Weather forecasting using Hidden Markov Model," 2017 *International Conference on Computing and Communication Technologies for Smart Nation (IC3TSN)*, 2017, pp. 220-225, doi: 10.1109/IC3TSN.2017.8284480.
- [24] A. Adel and A. El Mougy, "Cloud Computing Predictive Resource Management Framework Using Hidden Markov Model," 2022 5th *Conference on Cloud and Internet of Things (CIoT)*, 2022, pp. 205-212, doi: 10.1109/CIoT53061.2022.9766809.
- [25] C. Xue, "A Novel English Speech Recognition Approach Based on Hidden Markov Model," 2018 *International Conference on Virtual Reality and Intelligent Systems (ICVRIS)*, 2018, pp. 1-4, doi: 10.1109/ICVRIS.2018.00009.
- [26] Viterbi, A.J., 2006. A personal history of the Viterbi algorithm. *IEEE Signal Processing Magazine*, 23(4), pp.120-142.
- [27] Kawazu, H., Toriumi, F., Takano, M., Wada, K. and Fukuda, I., 2016, December. Analytical method of web user behavior using Hidden Markov Model. In *2016 IEEE International Conference on Big Data (Big Data)* (pp. 2518-2524). IEEE.
- [28] Chuang, K.C., Shih, C.S. and Hung, S.H., 2011, November. User behavior augmented software testing for user-centered gui. In *Proceedings of the 2011 ACM Symposium on Research in Applied Computation* (pp. 200-208).
- [29] Yu, L., Zhang, W., Wang, J. and Yu, Y., 2017, February. Seqgan: Sequence generative adversarial nets with policy gradient. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 31, No. 1), 2017, pp. 2852-2858.
- [30] Aktas, M. S., & Kapdan, M. (2016). Structural code clone detection methodology using software metrics. *International Journal of Software Engineering and Knowledge Engineering*, 26(02), pp. 307-332.
- [31] M. Kapdan, M. Aktas and M. Yigit, On the structural code clone detection problem: A survey and software metric based approach, in *Int. Conf. Computational Science and its Applications*, 2014, pp. 492-507.
- [32] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kegl. Algorithms for hyper-parameter optimization. In *Proc. of NIPS'11*, pp. 2546-2554, 2011.
- [33] Yu T, Zhu H. Hyper-parameter optimization: A review of algorithms and applications. *arXiv preprint arXiv:2003.05689*. 2020 Mar 12.
- [34] Feurer M, Hutter F. Hyperparameter optimization. In *Automated machine learning 2019* (pp. 3-33). Springer, Cham.